



DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute affiliated to Visvesvaraya Technological University (VTU), Belagavi,
Approved by AICTE and UGC, Accredited by NAAC with 'A' grade)
Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-560111



DEPARTMENT OF INFORMATION SCIENCE & ENGINEERING

(Accredited by NBA Tier 1: 2025-2028)

Alternate Assessment Tool - 2 “Scenario Investigation”

Agile methodologies in an organization transitioning from a traditional waterfall approach

Report submitted for the subject *INTRODUCTION TO AGILE-DEVOPS INTEGRATION – 22IS673*

Submitted by

BHOOMIKA K NAYAK (1DS23IS031)

SEMESTER: VI

GROUP – A08

Under the Guidance of

Dr. Reshma J

Associate Professor

Department of Information Science and Engineering
DSCE, Bengaluru

**VISVESVARAYA TECHNOLOGICAL UNIVERSITY
JNANASANGAMA, BELAGAVI-590018, KARNATKA
2025-26**

DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute affiliated to Visvesvaraya Technological University (VTU), Belagavi,
Approved by AICTE and UGC, Accredited by NAAC with 'A' grade)
Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-560111

DEPARTMENT OF INFORMATION SCIENCE & ENGINEERING

(Accredited by NBA Tier 1: 2025-2028)



CERTIFICATE

This is to certify that the Alternate Assessment Tool (AAT) - 2 submitted as part of **INTRODUCTION TO AGILE-DEVOPS INTEGRATION (22IS673)** is a bonafide work carried out by **Bhoomika (1DS23IS031)** as 10-mark component in partial fulfillment for the 6th Semester of Bachelor of Engineering in Information Science & Engineering of the Visvesvaraya Technological University, Belagavi during the year 2025-26. The AAT report has been approved as it satisfies the academic requirements prescribed for the Bachelor of Engineering degree.

Signature of Faculty
[Dr.Reshma J]

Signature of HOD
[Dr.Annapurna P Patil]

DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute affiliated to Visvesvaraya Technological University (VTU), Belagavi,
Approved by AICTE and UGC, Accredited by NAAC with 'A' grade)
Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-560111

DEPARTMENT OF INFORMATION SCIENCE & ENGINEERING

(Accredited by NBA Tier 1: 2025-2028)



DECLARATION

I declare that I abide by the ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice. The work submitted in this report of INTRODUCTION TO AGILE-DEVOPS INTEGRATION – 22IS673, VI Semester BE, ISE has been compiled by referring to the relevant online and offline resources to the best of my understanding and in partial fulfillment of the requirement for the award of the degree of Bachelor of Engineering in Information Science & Engineering, at Dayananda Sagar College of Engineering, an autonomous institution affiliated to VTU, Belagavi during the academic year 2025-26.

I hereby declare that the same has not been submitted in part or full for other academic purposes.

Bhoomika K Nayak (1DS23IS031)

Place: Bangalore

Date: 21-04-2026

Contents

Introduction	1
Risks of Transitioning from Waterfall to Agile	1-3
Impact on Project Timelines	3
Impact on Team Dynamics	3-4
Impact on Product Quality	4-5
Mitigation Strategies	5-7
Conclusion	7-8
Reference	8

1.Introduction

The transition from a traditional Waterfall methodology to an Agile framework represents one of the most significant organizational shifts a software development team can undertake. Waterfall, characterized by its sequential phases—requirements, design, implementation, verification, and maintenance—has long been the standard for managing large-scale projects with well-defined scopes. Agile, on the other hand, emphasizes iterative development, customer collaboration, and adaptability to change.

While Agile offers compelling advantages such as faster delivery cycles, improved customer satisfaction, and better responsiveness to changing requirements, the transition itself is fraught with risks. These risks, if not carefully managed, can derail project timelines, fracture team cohesion, and compromise overall product quality. This report examines the key risks associated with implementing Agile methodologies in organizations moving away from Waterfall, analyzes their impact across three critical dimensions—project timelines, team dynamics, and product quality—and proposes comprehensive mitigation strategies.

2. Risks of Transitioning from Waterfall to Agile

2.1 Cultural and Mindset Resistance

One of the most pervasive risks during an Agile transition is organizational culture resistance. Employees and management who have operated under Waterfall for years are accustomed to clearly defined roles, hierarchical decision-making, and long-range planning horizons. Agile demands a fundamentally different mindset—one that embraces uncertainty, values collaboration over documentation, and treats change as an opportunity rather than a threat.

Resistance manifests in multiple ways: developers may resist the overhead of daily stand-ups and retrospectives; managers may struggle to relinquish control over detailed project plans; and business stakeholders may be uncomfortable with the absence of fixed-scope contracts. This cultural inertia can render Agile adoption superficial—teams go through the motions of Scrum ceremonies without internalizing the underlying principles, a phenomenon often called "Agile in name only."

2.2 Lack of Agile Experience and Training

Teams accustomed to Waterfall often lack the practical knowledge required to execute Agile effectively. Concepts such as sprint planning, backlog grooming, velocity tracking, story point estimation, and the role of a Product Owner versus a Scrum Master are often misunderstood or

misapplied. Inadequate training leads to poorly estimated sprints, incomplete user stories, and Scrum Masters who function more as project managers than servant-leaders.

Without experienced Agile coaches or certified practitioners guiding the transition, organizations frequently misapply frameworks such as Scrum or Kanban, leading to confusion about roles and responsibilities and undermining the intended benefits of the methodology.

2.3 Scope Creep and Unclear Requirements

Agile's flexibility—its openness to changing requirements even late in development—is both its greatest strength and a significant risk vector. Without disciplined backlog management and a strong Product Owner, this flexibility can devolve into uncontrolled scope creep. Stakeholders, aware that Agile accommodates changes, may continuously introduce new features, alter priorities mid-sprint, or fail to make definitive decisions, leading to an ever-expanding backlog that never converges.

In Waterfall, the requirements phase produces a fixed specification document that acts as a contract. Agile removes this safety net, requiring ongoing negotiation and prioritization that many organizations are not initially equipped to manage effectively.

2.4 Inadequate Infrastructure and Tooling

Agile and DevOps practices are deeply intertwined. Effective Agile delivery typically depends on Continuous Integration/Continuous Delivery (CI/CD) pipelines, automated testing frameworks, version control systems, and collaborative project management tools. Organizations transitioning from Waterfall often lack this infrastructure. Manual testing regimes, monolithic codebases, and the absence of automated build systems make it difficult to deliver working software at the end of every sprint as Agile prescribes.

The investment required to modernize infrastructure often surprises organizations mid-transition, creating bottlenecks and slowing the pace of Agile adoption precisely when velocity and momentum are most needed.

2.5 Misalignment Between Agile Teams and Non-Agile Stakeholders

In many organizations, particularly large enterprises, software development teams may adopt Agile while the broader organization—including procurement, legal, finance, and executive leadership—continues to operate under Waterfall assumptions. This creates a fundamental disconnect. Agile teams produce working software in two-week sprints, but executive reporting

still demands quarterly Gantt charts. Procurement contracts are signed on fixed-scope, fixed-price models that conflict with Agile's adaptive nature.

This organizational misalignment is a systemic risk that cannot be resolved at the team level alone and requires executive sponsorship and enterprise-wide transformation efforts.

3. Impact on Project Timelines

The risks identified above exert significant pressure on project timelines during the Agile transition period. Cultural resistance and inadequate training lead to inefficient sprint ceremonies, poor backlog refinement, and inaccurate sprint velocity estimates. Teams routinely fail to complete sprint goals, leading to carryover items that disrupt subsequent sprints and create a cascading delay effect.

Scope creep directly extends project timelines. When stakeholders continuously introduce new requirements or change priorities mid-sprint, the definition of 'done' shifts constantly, making it impossible to converge on a releasable product. Studies in software project management have consistently shown that poorly managed Agile transitions can result in timeline overruns equivalent to or worse than traditional Waterfall projects, because teams carry the overhead of Agile ceremonies without realizing their benefits.

The absence of automated testing and CI/CD pipelines—a consequence of inadequate tooling—forces teams to rely on manual quality assurance, which is slow and error-prone. This creates a bottleneck at the end of each sprint, often pushing testing into a dedicated hardening sprint that undermines the Agile principle of continuous shippability.

Furthermore, organizational misalignment forces Agile teams to produce dual artifacts: Agile-native sprint boards for the development team and traditional project plans and status reports for executive stakeholders. This administrative overhead consumes development capacity and slows delivery cycles.

4. Impact on Team Dynamics

The transition to Agile profoundly disrupts established team structures and interpersonal dynamics. In Waterfall environments, roles are clearly delineated—business analysts write requirements, architects produce designs, developers code, and testers validate. Agile requires these boundaries to blur. Cross-functional teams are expected to collectively own the full software

delivery lifecycle, which can cause identity conflicts among specialists who define their professional value through role-specific expertise.

Resistance from experienced senior engineers is particularly common. Engineers who have built their careers around deep specialization may resist the expectation to participate in sprint planning, write user stories, or engage in retrospectives they perceive as managerial overhead. Conversely, junior team members may feel overwhelmed by the expectation of shared ownership without adequate mentorship structures.

The introduction of Scrum ceremonies—daily stand-ups, sprint reviews, retrospectives—can initially feel intrusive to teams unaccustomed to such transparency. Stand-ups may degenerate into status meetings; retrospectives may become blame sessions if psychological safety is not actively cultivated. These dysfunctions erode trust and reduce team cohesion precisely at the moment when cohesion is most critical.

Management dynamics are equally affected. Traditional project managers, whose authority derived from controlling information and producing detailed plans, must reinvent themselves as servant-leaders or risk becoming redundant. Those who fail to make this transition often create parallel command structures that undermine Scrum Masters and Product Owners, producing conflicting priorities and decision-making paralysis.

5. Impact on Product Quality

Product quality is perhaps the most immediately measurable casualty of a poorly managed Agile transition. The sprint model's time-boxing creates pressure to deliver working software every two weeks. Without a strong culture of technical excellence—including test-driven development, continuous integration, and refactoring—teams sacrifice quality for velocity. Technical debt accumulates rapidly when 'done' is defined as 'functional' rather than 'tested, refactored, and production-ready.'

Inadequate test automation means that regression testing is deferred or abbreviated, allowing defects introduced in one sprint to go undetected until later sprints or, worse, until production. The cost of detecting and fixing defects grows exponentially with time, and in an Agile context where codebase complexity grows sprint-by-sprint, unmanaged technical debt can eventually freeze velocity entirely.

Unclear user stories and poor backlog refinement directly impact product quality by leading to features that satisfy acceptance criteria on paper but fail to meet actual user needs. When Product Owners are inexperienced or unavailable, developers make assumptions about requirements that result in rework, misaligned features, and user dissatisfaction—outcomes antithetical to Agile's customer-centric promise.

6. Mitigation Strategies

6.1 Invest in Agile Training and Coaching

The most foundational mitigation is ensuring all team members—developers, testers, Product Owners, Scrum Masters, and relevant stakeholders—receive structured Agile training before and during the transition. Organizations should engage experienced Agile coaches who embed with teams to provide real-time guidance rather than delivering classroom training and disengaging. Certifications such as Certified Scrum Master (CSM), Professional Scrum Product Owner (PSPO), and SAFe certifications for enterprise contexts provide a common vocabulary and shared mental models that reduce friction.

- Conduct Agile workshops tailored to each role before the first sprint.
- Engage external Agile coaches for the initial 3–6 months of the transition.
- Create internal communities of practice where practitioners share lessons learned across teams.

6.2 Adopt a Phased Transition Approach

Rather than performing a 'big bang' transition where the entire organization switches to Agile overnight, organizations should adopt a phased approach. Begin with one or two pilot teams on projects that are suitable for Agile—those with evolving requirements and active stakeholder engagement. Use the pilot to refine the adoption model, identify organization-specific challenges, and demonstrate the value of Agile through measurable outcomes before scaling.

Frameworks such as the Scaled Agile Framework (SAFe) or Large-Scale Scrum (LeSS) are designed specifically for phased enterprise adoption and provide structural guardrails for managing the complexity of scaling Agile across multiple teams while preserving organizational continuity.

- Select low-risk, high-visibility pilot projects that can demonstrate early wins.
- Use pilot retrospectives to capture lessons that inform the broader rollout.
- Establish an Agile Center of Excellence (CoE) to govern and support scaling.

6.3 Strengthen Backlog Management and Product Ownership

To mitigate scope creep and requirements ambiguity, organizations must invest in developing strong Product Owner capabilities. Product Owners must be empowered with decision-making authority, have deep domain knowledge, and be available to the development team throughout each sprint. Definition of Ready (DoR) criteria—standards that user stories must meet before entering a sprint—provide a quality gate that prevents poorly specified work from entering development.

- Establish a Definition of Ready: user stories must have clear acceptance criteria, be independently testable, and be estimated before sprint planning.
- Conduct bi-weekly backlog refinement sessions to groom upcoming stories.
- Train Product Owners in story mapping, impact mapping, and prioritization frameworks such as MoSCoW and WSJF (Weighted Shortest Job First).

6.4 Build DevOps and Automation Infrastructure

Sustainable Agile delivery requires an engineering foundation that supports rapid, reliable releases. Organizations must invest in building CI/CD pipelines, automated unit and integration test suites, and infrastructure-as-code capabilities in parallel with the Agile adoption. Jenkins, GitHub Actions, GitLab CI, and Azure DevOps are widely used platforms that enable automated build, test, and deployment workflows that align naturally with sprint cadences.

- Establish a CI/CD pipeline as a prerequisite before the first sprint begins.
- Set a target test automation coverage threshold (e.g., 80% unit test coverage) and track it as a sprint metric.
- Adopt infrastructure-as-code tools such as Terraform or Ansible to enable reproducible, automated environment provisioning.

6.5 Cultivate Psychological Safety and Team Culture

Agile retrospectives are the primary mechanism for continuous team improvement, but they are only effective in psychologically safe environments where team members feel comfortable surfacing problems without fear of blame. Leadership must actively model vulnerability and openness to feedback. Scrum Masters should be trained in facilitation techniques that encourage honest dialogue and focus retrospectives on systemic process improvements rather than individual performance.

- Use structured retrospective formats such as the 4Ls (Liked, Learned, Lacked, Longed For) or the Starfish model to ensure balanced feedback.
- Establish team working agreements that define norms for communication, conflict resolution, and decision-making.
- Celebrate sprint achievements publicly to reinforce the positive impact of Agile practices.

6.6 Align Executive and Organizational Governance

Organizational misalignment between Agile teams and Waterfall-oriented stakeholders requires deliberate governance redesign. Executive leadership must be educated on Agile reporting models—using metrics such as team velocity, sprint burn-down, release burn-up, and cumulative flow diagrams instead of traditional Gantt charts. Portfolio management should shift from project-based to product-based thinking, with funding allocated to product teams rather than discrete projects.

- Replace quarterly project status reports with product roadmap reviews and release burn-up charts.
- Restructure procurement and contracts to accommodate Agile delivery models, including time-and-materials contracts with defined sprint cadences.

Engage executives as active participants in sprint reviews to foster direct visibility into delivered value.

7. Conclusion

The AI-driven Civic Triage System addresses the key challenges faced by traditional complaint management systems by introducing automation, intelligence, and efficiency into the process. By integrating advanced technologies such as Natural Language Processing, Computer Vision, and Speech Recognition, the system is capable of handling multimodal inputs and extracting meaningful information from diverse data sources. This enables accurate classification of complaints and ensures that they are routed to the appropriate departments without delays.

The system significantly reduces manual effort and minimizes errors by automating tasks such as complaint analysis, urgency detection, and geo-aware routing. The inclusion of priority levels helps authorities focus on critical issues, improving response time and enhancing public safety. Additionally, features like duplicate detection prevent redundancy and optimize resource utilization, making the system more efficient.

Another important aspect of the system is its ability to improve transparency and user satisfaction. Citizens can track the status of their complaints, which builds trust and encourages active participation in governance. The system also supports human-in-the-loop decision-making, ensuring that final actions are taken by responsible authorities while using AI as a supporting tool.

Overall, the proposed system provides a scalable and reliable solution for modern urban governance. It enhances service delivery, reduces delays, and improves the overall quality of civic management. In the future, the system can be further enhanced by integrating real-time data streams, advanced predictive analytics, and smart city technologies, making it more robust and adaptable to evolving urban needs.

8. References

- [1] Beck, K., et al. (2001). Manifesto for Agile Software Development. <https://agilemanifesto.org/>
- [2] Cohn, M. (2009). Succeeding with Agile: Software Development Using Scrum. Addison-Wesley Professional.
- [3] Kim, G., Humble, J., Debois, P., & Willis, J. (2016). The DevOps Handbook. IT Revolution Press.
- [4] Leffingwell, D. (2010). Agile Software Requirements: Lean Requirements Practices for Teams, Programs, and the Enterprise. Addison-Wesley.
- [5] Larman, C., & Vodde, B. (2016). Large-Scale Scrum: More with LeSS. Addison-Wesley Professional.
- [6] Scaled Agile Framework (SAFe). <https://scaledagileframework.com/>
- [7] Schwaber, K., & Sutherland, J. (2020). The Scrum Guide. <https://scrumguides.org/>
- [8] Fowler, M. (2006). Continuous Integration.
<https://martinfowler.com/articles/continuousIntegration.html>